

Lecture 3 — The Execution Model

Jeff Zarnett

2026-08-29

So You Want to Run a C Program

To deeply understand how a process executes within the operating system, we have to start with making a program. It's an essential part of a process, after all, and how we build one matters for how it will run when the time comes. Then, a program needs to be loaded (and the process created) to start executing. We'll look at compiling & linking, loading a program (including the ELF format), and talk about stack frames and calling conventions when executing, and a little about the how asynchronous execution works, too.

Compilation & Linking. Our experience with writing C programs is that they are compiled: there is a tool, the compiler, that turns the code that appears in the source files into something the computer can understand. That output is the executable code and some associated data, like we discussed when defining a process.

Many of the programs we write in a university context are small enough that the source all fits one file and maybe a header file. A real-world, large software project involves a lot more code than is going to fit in a single source file. If that's the case, we need to connect the different compiled output files to build the program. It's permissible for function `A()` in one file to call function `B()` in another file, where those two files get compiled into separate units. When compiling the unit that contains `A()`, the compiler must leave a note that says “when we figure out where `B()` is, put its address here” so that the call will actually work when the program runs.

We also spent extensive time in ECE 252 doing things that used libraries such as the `pthread` library, the standard C library. The library code does not get copy-pasted into the source of the executable; instead the binary you wrote references the library as it exists in the system. Here, the compiler, again, leaves a slightly different kind of note saying “at load time, figure out what the address of `write()` is, and put its address here” so that system call wrapper will work as intended.

Side note: you *can* copy-paste the library code into the executable. That's called *static linking* and while it's something that it is possible to do, it is not recommended in many situations [Dre]. With that said, languages like Go do this. Static linking makes the binary file bigger than it needs to be, sure, but it also means it is easy for a security vulnerability or performance bug to persist in your program. To fix such a vulnerability, you have to recompile, re-link, and then ship the changed binary. The alternative is *dynamic linking*, and that looks a lot more like “use the version of this library that you find on the system already”. That's not perfect either – you can break many programs at once with a problem in the library – but we usually prefer dynamic linking either way.

The part of going back and filling in those pieces, or building the connections between the different parts of the program, is the post-compilation step called *linking*. Linking brings the different parts of the program together to create a cohesive program by resolving all those “we'll figure this out shortly” notes that it has left for its future self. There's compile-time linking for the first kind of note, where the location of `B()` is figured out at compile time and assigned. There is also dynamic linking, which is for the second kind of note, figuring out where `write()` is, and that happens later, when launching the process.

You experience a compile-time linking error any time you mistype a function name and the compiler says it has an undefined reference to function `do_tihng()` when clearly you meant it to be `do_thing()`. The compile-time linker does need to be assured of the existence of the library functions that appear in the second kind of note, otherwise it will say it does not know what function is being referenced. Only if the linker is able to resolve all notes of both kinds is there a valid executable produced.

Loading. Given a valid executable that we want to run (by request of the user or as the result of a fork-exec flow), we need to take the program and turn it into a process.

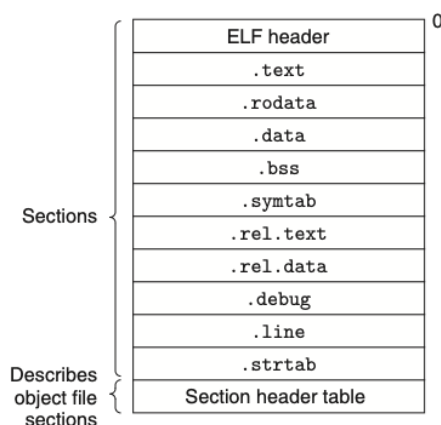
The concept of the process and process control block was mentioned in ECE 252 as part of understanding the backdrop, and revisited in the previous topic. In both places, the content focused on recognizing the difference between a program and a process and on what information about a process the operating system must know (and storing that in a data structure). This skips over the details about how to actually get the executable program into a state where it can run. That's what we cover here.

As we know, `fork()` makes an exact copy of a running process (with a new process control block and the like). We also covered the idea of `execve()` and its variants as the system call necessary to make a child process do something other than run the same code as its parent process. The `execve()` system call steps walk us through all the things necessary to turn a program that is not running into a running process.

The first thing that happens is throwing away all the things that are to be replaced: the executable program, memory, and other resources (with exceptions) all go away. Some things survive, like the process ID and even open file descriptors. After that, it is a blank canvas. So the OS can then load the compiled executable and start reading its contents to figure out how to set it up. Indeed, the compiled executable comes with a little setup guide, akin to IKEA instructions, for how to build its initial state. The OS should read them. Even if it is really good at building Kallax units after doing so like ten times.

Consistent with other topics, for time reasons, the focus is on UNIX/Linux. The concrete examples will be using the Linux Executable and Linkable Format (ELF) file format (as described in [BO15]). Windows, macOS, and other operating systems have their own formats which will differ in some ways. The fundamentals are the same across each of them, though, so the concepts carry over just fine.

Executable Format. The different parts of the program that need to be put together are *sections*. Consider the ELF format shown below, that we will walk through to understand each section; the graphic as well as the explanation list of each section are from the same source [BO15]:



- The **ELF Header** explains the word size and byte order of the system that made it, and how to interpret the rest of the file. An introduction, if you will.
- The `.text` section contains the compiled binary code of the program.
- The `.rodata` section contains read-only data like format strings.
- The `.data` section contains global and static variables if initialized.
- The `.bss` section is a placeholder; uninitialized global variables or things initialized to zero go here but take no space in the file.

- The `.symtab` section is the symbol table, that explains what functions and global variables are defined in the program.
- The `.rel.text` section is locations that need to be modified by the linker: these are notes mentioned earlier to come back to later.
- The `.rel.data` section is relocation information for global variables.
- The `.debug` section is debugging information if the compiler arguments asked to generate it.
- The `.strtab` section is a string table for the things in `.symtab` and `.debug` sections and section names.
- The **Section header table** is a table of contents for each section in the file.

There is much more complexity to how a program is linked against libraries and other dynamically-decided locations than we will have time for right now. Four sections need to get loaded into memory for the process to get started: `.text`, `.rodata`, `.data`, `.bss`. The two other sections that start with `.rel` are used at this time to resolve some more of the “notes” we talked about earlier.

Each of the four sections to be loaded needs to be put at some memory address. Exactly where can vary – as we’ll see in a bit – but they have to be in memory to run. The four are meant to be loaded with permissions; i.e., the `.text` is executable but cannot be written, to prevent accidentally changing the executable code.

Of course, before we can start, we also need to put the stack and heap into memory as well. These start out as just regions of memory that contain whatever garbage was present in RAM, and then they are available as areas for running the process. Remember that these are not allocated with something like `malloc()`; that’s how a user-space program asks to get more heap memory. From the operating system’s point of view, it can simply declare an area of memory to be the heap for a given process. It didn’t say it, it declared it.

The process starts with just the one thread running, but the process can ask for new threads to be created. That happens after the process has started running, of course, and will require a new stack for each new thread created. The basic idea is the same, though, it just happens later.

And once we’ve got things loaded, it’s possible to call the process ready to run! Execution can begin when the scheduler chooses that process. Execution actually starts slightly before `main()` with some setup work (and there’s a corresponding cleanup after the end of `main()`) but once all the setup is done we’re ready to go.

Calling Conventions. The program counter register points to the next statement that the CPU is going to execute, which is necessary but not sufficient to represent the state of execution of the code. If a program is all written inside of `main()` and we just execute sequentially (with loops and conditional statements) then we might not need it, but when we have function calls (procedures) in the program, we need a way of keeping track of how we got here: when function `compute()` calls function `load()`, we use the stack to keep track of our execution state, so that the end of the `load()` function successfully returns the result back to the caller.

ECE 252 emphasized that when writing C code, knowing the difference between what is on the stack and what is on the heap means that you can manage memory correctly and avoid memory leaks, crashes, and other bad outcomes. Variables local to a function appear in the stack (or in registers) whereas things allocated dynamically with `malloc()` are in the heap. The stack used for program execution is the same one where those local storage variables exist.

When a function call to start the `load()` function takes place, a new *stack frame* must be created. In addition to the local storage for that function, the stack frame is used for passing arguments, storing the return information, and is used to save registers. Whether the stack frame grows towards higher or lower addresses is a design decision and convention; it does not really matter for this discussion. The x86 architecture has the stack growing towards lower addresses. When the currently-executing function returns, the stack frame for it is popped off the stack. As you would expect from C, doing so does not erase the contents of it; that memory is just viewed as unused.

A function being called often has arguments. The arguments need to be provided and there are two viable ways: registers and the stack. The 32-bit architecture pushed all the parameters onto the stack – this was covered back in ECE 252’s setup of how to use the system call of `read()`, if you recall – and that is certainly convenient because we can put an arbitrary number of arguments there. Good program design may discourage a 95-argument function, but nothing about the architecture forbids it.

In the 64-bit x86 architecture, registers enter the picture. This is desirable because they are faster than putting things in main memory, but they are limited in space. The architecture convention provides for 6 word-size (64 bit) registers; if that’s not enough then any extra arguments go to the stack [BO15]. There’s some complexity about how these registers are filled; there’s a specific order for filling them. If a type is too big for a single register, like a 128-bit type, it can consume two registers. If a type is smaller than word size, like a 32-bit integer, it still uses the full register and “wastes” a little space.

Which registers are available for parameter passing is a design decision of the platform. Other systems, such as the 32-bit STM boards used in MTE 241, have a different set of rules.

If function `A()` is calling `B()`, the registers for `A()` are likely being used for something, either its own arguments or local variables. Therefore, `A`’s values need to be saved and restored later. There is further complexity about which registers are saved by the caller and which are saved by the callee, but getting a list of exactly which is getting down to the level of trivia. The need for both caller and callee to save and restore, or at least not change, certain registers increases the cost of using registers to pass the data, and also explains why “only” six registers are used for this purpose in the x86-64 architecture; if more registers were used for this purpose, it would result in more work to save and restore them.

Once the setup of the new stack frame is done, control needs to move there. That is, update the program counter to point to the start of function `B()` when it is time to run it. To do that, the assembly instructions to go to that destination not only transfers control, but it also writes down the return location.

The return information includes where execution goes back to, when the current function ends. Imagine the function call looks like `int b = get_open_balance();`. Inside that function, putting `return 0;` at the end of the function sends 0 back to the caller. Execution then resumes in the parent, with the next part being assigning the value of `b` and then the next statement in the program¹. To send the return value back, the value will go in a register if it fits; otherwise, a little workaround is needed.

Side Track: Stack Smashing Attacks. Given that the return location is just sitting there in the stack frame, it is vulnerable to malicious actors. Recall from the introduction to security that a goal that an attacker may have is to redirect the control flow of the program to a location of their choosing. That could be some already-existing part of the program, or to arbitrary code that they have written themselves and loaded into memory.

One way that a malicious actor can do this is a *stack smashing* attack. The goal here is to write too much data into a stack-allocated buffer. That works only if the code itself does not do adequate validation and limitation of the length of data written, but as we know from hard-won experience, string handling in C is difficult to get consistently correct. But, if the malicious actor is able to write past the normal end of the buffer, they can overwrite the return address and send program execution to whatever address they wrote there. In some cases, this is another part of the program; or it could be compiled code that they put into the buffer as part of overflowing the buffer to begin with.

Of course, it is possible to write programs in such a way that stack smashing attacks are no longer a risk, and program authors absolutely should. However, the operating system has three mitigation strategies that can be applied to stop this from happening even if a vulnerable program continues to run.

The first is called *address space randomization*. This happens when the process is being loaded: the base address of different parts of memory, including the stack and program executable code has a randomized starting address. That makes it very difficult to guess where in memory anything is going to be. If the attacker is loading their own

¹Okay, maybe the compiler can take a shortcut here where the return value goes directly into `b` but that’s not the point that matters here.

code into the buffer and trying to send the execution there, they need to know where it is; if it's another part of the program executable code, they need to find that, too. Changing the location on every run of the program defeats a guess-and-check kind of strategy.

The second strategy is called a *stack canary*. The stack canary is a randomly-generated value that is placed in between the local variables and the return address in the stack frame. When the function returns, this value is checked, to see if it has changed and stop execution if so. If an attacker successfully overwrites the buffer and gets to the return address, they will have necessarily overwritten the canary, and because the canary value is randomly generated, that also makes the guess-and-check strategy infeasible here as well.

The third strategy is marking some memory as no-execute (NX for AMD, or XD, execute-disable for Intel), which is a processor feature. This is used to designate parts of memory that are meant to be for data, not executable code, as ineligible to be executed. If the CPU is redirected to execute an instruction that is in a memory area that is designated as NX, the processor rejects it. That's enforcement of the permissions for the different sections mentioned above.

These mitigations specifically stop the redirection of control flow to the attacker's desired location. It is still possible to use a stack-smashing attack to crash the process by changing the return address to something invalid. This can be used to execute a denial-of-service attack, though, by repeatedly crashing the processes.

Interrupts. In ECE 252 we discussed interrupts, starting with sending signals. The canonical first example introduces a handler for SIGINT that changes a boolean quit to be true which means on the next iteration of the loop, the program will exit and clean up gracefully. The thing that made signals exciting is that some of them, including SIGINT, are asynchronous: they can come in at any time and are not tied to any particular line of code. Assuming the signal is not deferred or ignored, it interrupts what we are doing and runs the signal handler.

ECE 252 also said explicitly that there are limitations on what functions may safely be invoked in a signal handler: ones that are designated *async-signal safe* in their documentation. Often, this has to do with not messing up library state (e.g., trying to allocate memory while in the middle of another memory allocation).

As previously discussed, there are also synchronous signals. A division by zero or segmentation fault is synchronous, because you can point to a specific line of code where it happened. The last way that we covered for how a signal can be triggered is the `kill()` or `raise()` functions. No matter where the signal came from, handling it is the same.

Recall that the signal is delivered by the operating system to the process. For the process to execute the signal handler, there must exist a context for it: a stack frame. To run that, the kernel will push a stack frame onto the user stack. The signal handler will run in that stack frame. The frame is set up so that at the end of the handler, control is sent to little block of code that asks the operating system to unwind the signal frame by invoking the system call `sigreturn()`. That will put things back to where they were before the signal took place and execution will resume.

Signals are not the only kind of interrupt, of course. Many interrupts arise from hardware and are handled by the kernel with its own interrupt handling code. The kernel may choose, if appropriate, to use signals as the way of notifying the user process of what happened. If the user closes the lid of the laptop, this generates a hardware event that notifies the operating system that it should go to sleep, which will activate the relevant functionality and put everything to sleep.

Even a SIGINT has some hardware interrupt behind it, because it often starts with the user pressing Ctrl-C on the keyboard. That keyboard input goes to the kernel for handling and is then delivered to the user process that needs to receive it.

Hardware interrupt handlers do not end with normal return statements, but often something like `rti` or `iret`. Think of it as "return to execution". The return statement does not work; its semantics (as discussed earlier) terminate the current function and return control to the calling function. An interrupt handler should instead send execution back to where it was before the interrupt took place. There is no calling function to go back to.

Life Cycle. We have a reasonably good idea of how to run a program and how it executes. That only works, however, when the operating system itself is running. The next thing we should consider is how do we get the operating system to that state? Let us consider a day in the life of the operating system next.

References

- [BO15] Randal E. Bryant and David R. O'Hallaron. *Computer Systems: A Programmer's Perspective*. Pearson, 3rd edition, 2015.
- [Dre] Ulrich Drepper. Static linking considered harmful. https://www.akkadia.org/drepper/no_static_linking.html. Online; accessed 27-August-2026.